

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Кафедра экономической информатики

Дисциплина: Программирование сетевых приложений

**Отчёт о выполнении
лабораторной работы №1**
Вариант №6

Выполнял:
Биза Г.А.
гр. 414302

Проверил:
Иномов С. Ф.

Минск 2026

Цель работы: научиться создавать консольные приложения на Java, освоить принципы структурирования кода, обработки ввода/вывода, исключений и логирования с использованием библиотеки Log4j.

Задачи:

- изучить структуру консольного приложения Java;
- реализовать ввод и вывод данных через консоль;
- освоить обработку исключений;
- подключить и настроить библиотеку Log4j;
- реализовать логирование событий, ошибок и отладочной информации;
- выполнить индивидуальное задание с использованием логирования.

Теоретические сведения

Консольное приложение – программа, которая взаимодействует с пользователем через текстовый интерфейс: ввод с клавиатуры и вывод в консоль.

Java – строго типизированный язык, и каждая переменная имеет тип. Переменные – именованные области памяти. Основные типы переменных:

- int – целое число;
- double – число с плавающей запятой;
- boolean – логическое значение;
- char – символ;
- String – строка.

Массив – структура данных, хранящая фиксированное количество элементов одного типа.

Ввод и вывод данных – основа взаимодействия пользователя с программой. В Java для ввода используется класс Scanner, а для вывода – System.out.

Для чтения данных с клавиатуры используется класс Scanner.

Исключения – события, нарушающие нормальное выполнение программы. Обработка исключений позволяет избежать аварийного завершения и обеспечить устойчивость приложения. К ключевым конструкциям относятся блоки: try, catch и finally.

Логирование – процесс записи событий, ошибок и действий программы. Оно помогает отслеживать поведение приложения, выявлять ошибки и анализировать производительность.

Log4j – популярная библиотека для логирования в Java. Она позволяет записывать информацию о работе программы в консоль или файл, классифицируя сообщения по уровням важности. Уровни логирования:

- DEBUG – подробная отладочная информация;
- INFO – общая информация о ходе работы;
- WARN – предупреждения;
- ERROR – ошибки, не приводящие к завершению;
- FATAL – критические ошибки.

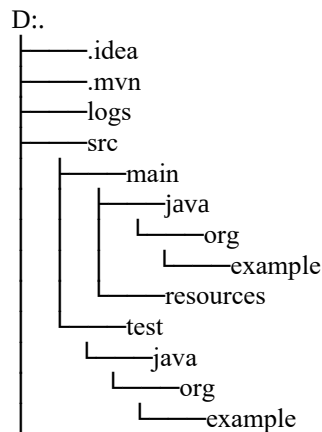
Методы – блоки кода, выполняющие определённую задачу. Они повышают читаемость и переиспользуемость.

Хорошо структурированный код облегчает сопровождение, тестирование и повторное использование. В консольных приложениях важно разделять:

- ввод/вывод;
- логика обработки;
- логирование;
- обработка ошибок.

Ход выполнения работы

Структура проекта организована по примеру из методического пособия и выглядит следующим образом:



Чтобы можно было использовать логирование, нужно подключить библиотеку Log4j через Maven (в pom.xml файле). В нём нужно прописать лишь данные строки кода, после чего обновить Maven через Reload:

```
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-core</artifactId>
  <version>2.23.1</version>
</dependency>
<dependency>
  <groupId>org.apache.logging.log4j</groupId>
  <artifactId>log4j-api</artifactId>
  <version>2.23.1</version>
</dependency>
```

После этого нужно задать параметры для логирования в нашем консольном приложении. Через Log4j все события будут отображаться в консольном меню и записываться в отдельный файл, который называется app.log. В следующем фрагменте кода, показана настройка подключенной библиотеки:

```
rootLogger.level = info
rootLogger.appenderRefs = console, file
rootLogger.appenderRef.console.ref = Console
```

```

rootLogger.appenderRef.file.ref = File

appender.console.type = Console
appender.console.name = Console
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %d{HH:mm:ss.SSS} %-5level %logger{36} - %msg%n

appender.file.type = File
appender.file.name = File
appender.file.fileName = logs/app.log
appender.file.layout.type = PatternLayout
appender.file.layout.pattern = %d{HH:mm:ss.SSS} %-5level %logger{36} - %msg%n

```

Класс `Question` представляет собой модель тестового вопроса и отвечает за хранение и предоставление всех данных, связанных с одним заданием: текст вопроса (`question`), список вариантов ответов (`answers`), индекс правильного ответа (`rightAnswer`) и количество баллов за него (`points`). Основные функции класса включают инициализацию этих данных через конструктор, доступ к ним через геттеры и сеттеры, а также проверку ответа пользователя с помощью метода `checkAnswer()`, который сравнивает переданный индекс с правильным. Переопределенный метод `toString()` формирует удобочитаемое строковое представление вопроса с нумерацией вариантов. Класс обеспечивает структурированное хранение данных вопроса и инкапсулирует базовую логику его проверки.

```

package org.example;

import java.util.ArrayList;
import java.util.List;

public class Question {

    private String question;
    private List<String> answers = new ArrayList<>();
    private int rightAnswer;
    private int points;

    public Question(String question, List<String> answers, int rightAnswer,
int points){
        this.question = question;
        this.answers = answers;
        this.rightAnswer = rightAnswer;
        this.points = points;
    }

    public String getQuestion() {
        return question;
    }

    public void setQuestion(String question) {
        this.question = question;
    }

    public List<String> getAnswer() {
        return answers;
    }

    public void setAnswer(List<String> answer) {

```

```

        this.answers = answer;
    }

    public int getRightAnswer() {
        return rightAnswer;
    }

    public void setRightAnswer(int rightAnswer) {
        this.rightAnswer = rightAnswer;
    }

    public int getPoints() {
        return points;
    }

    public void setPoints(int points) {
        this.points = points;
    }

    public boolean checkAnswer(int userAnswer){
        return rightAnswer == userAnswer;
    }

    @Override
    public String toString() {
        StringBuilder sb = new StringBuilder();
        sb.append(question).append("\n");
        for(int i = 0; i < answers.size(); i++){
            sb.append(i+1).append(". ").append(answers.get(i)).append("\n");
        }
        return sb.toString();
    }
}

```

Класс `Task` отвечает за управление коллекцией вопросов, представляя собой отдельное тестовое задание или тему. Он содержит список объектов `Question` (`questions`), название задания (`taskName`), а также два поля для подсчета очков: набранные пользователем баллы (`userScore`) и максимально возможные баллы за все вопросы (`totalScore`). Основные функции класса: инициализация названия через конструктор, добавление нового вопроса методом `addQuestion()` (который автоматически увеличивает `totalScore` на баллы добавляемого вопроса), изменение счета пользователя через `addToUserScore()`, сброс счета методом `resetUserScore()`, а также предоставление доступа к данным через геттеры. Класс обеспечивает структурированное объединение вопросов в логические блоки и автоматический подсчет общей суммы баллов для последующего оценивания.

```

package org.example;

import java.util.ArrayList;
import java.util.List;

public class Task {
    private String taskName;
    private List<Question> questions = new ArrayList<>();
    private int userScore;
    private int totalScore;
}

```

```

public Task(String taskName){
    this.taskName = taskName;
    this.questions = new ArrayList<>();
    this.totalScore = 0;
    this.userScore = 0;
}

public void addQuestion(Question question){
    questions.add(question);
    totalScore += question.getPoints();
}

public void addToUserScore(int points){
    userScore += points;
}

public void resetUserScore(){
    userScore = 0;
}

public List<Question> getQuestions() {
    return questions;
}

public String getTaskName() {
    return taskName;
}

public int getUserScore() {
    return userScore;
}

public int getTotalScore() {
    return totalScore;
}
}

```

Класс 'TaskManager' является главным управляющим компонентом системы тестирования, отвечающим за инициализацию, хранение и проведение тестов, а также за логирование всех ключевых событий. Он содержит список заданий ('taskList'), сканер для ввода пользователя ('scanner') и статический логгер Log4j для отслеживания работы. Основные функции класса: автоматическое создание предустановленных тестов по истории, географии и литературе в методе 'initializeQuestions()' с логированием этого процесса; запуск тестирования через 'runTest()', который выводит доступные тесты, обрабатывает пользовательский ввод с проверкой ошибок и передает управление в 'takeTask()'; проведение выбранного теста с поочередным выводом вопросов, проверкой ответов через метод 'checkAnswer()' объекта 'Question', начислением баллов и детальным логированием каждого действия пользователя; подсчет и вывод результатов в 'showResults()' с расчетом процента правильных ответов и выставлением оценки, а также логирование итогов; отображение списка всех тестов и их вопросов через 'showAllTasks()'; и корректное закрытие ресурсов методом 'close()'. Класс обеспечивает полный цикл тестирования: от инициализации данных и взаимодействия с пользователем до подсчета результатов и логирования для последующего анализа.

```

package org.example;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class TaskManager {
    static Logger logger = LogManager.getLogger(TaskManager.class);
    private List<Task> taskList;
    private final Scanner scanner;

    TaskManager(){
        this.taskList = new ArrayList<>();
        this.scanner = new Scanner(System.in);

        initializeQuestions();
    }

    public void initializeQuestions(){
        logger.info("Initializing test questions");

        // Создаем тест по истории
        Task historyTask = new Task("История");

        List<String> answers1 = new ArrayList<>();
        answers1.add("1812 год");
        answers1.add("1917 год");
        answers1.add("1941 год");
        answers1.add("1991 год");
        historyTask.addQuestion(new Question(
            "В каком году началась Великая Отечественная война?",
            answers1,
            2, // правильный ответ: 1941 год (индекс 2)
            10
        ));

        List<String> answers2 = new ArrayList<>();
        answers2.add("Петр I");
        answers2.add("Екатерина II");
        answers2.add("Иван Грозный");
        answers2.add("Александр II");
        historyTask.addQuestion(new Question(
            "Кто основал Санкт-Петербург?",
            answers2,
            0, // правильный ответ: Петр I (индекс 0)
            15
        ));

        List<String> answers3 = new ArrayList<>();
        answers3.add("1240 год");
        answers3.add("1380 год");
        answers3.add("1480 год");
        answers3.add("1612 год");
        historyTask.addQuestion(new Question(
            "В каком году произошло Ледовое побоище?",
            answers3,
            0, // правильный ответ: 1240 год (индекс 0)
            15
        ));

        taskList.add(historyTask);
    }
}

```

```

// Создаем тест по географии
Task geographyTask = new Task("География");

List<String> answers4 = new ArrayList<>();
answers4.add("Москва");
answers4.add("Санкт-Петербург");
answers4.add("Новосибирск");
answers4.add("Казань");
geographyTask.addQuestion(new Question(
    "Какая столица России?",
    answers4,
    0, // правильный ответ: Москва (индекс 0)
    10
));

List<String> answers5 = new ArrayList<>();
answers5.add("Амазонка");
answers5.add("Нил");
answers5.add("Волга");
answers5.add("Миссисипи");
geographyTask.addQuestion(new Question(
    "Какая река является самой длинной в мире?",
    answers5,
    1, // правильный ответ: Нил (индекс 1)
    20
));

List<String> answers6 = new ArrayList<>();
answers6.add("Россия");
answers6.add("Канада");
answers6.add("Китай");
answers6.add("США");
geographyTask.addQuestion(new Question(
    "Какая страна имеет самую большую площадь?",
    answers6,
    0, // правильный ответ: Россия (индекс 0)
    15
));

taskList.add(geographyTask);

// Создаем тест по литературе
Task literatureTask = new Task("Литература");

List<String> answers7 = new ArrayList<>();
answers7.add("Война и мир");
answers7.add("Преступление и наказание");
answers7.add("Евгений Онегин");
answers7.add("Мертвые души");
literatureTask.addQuestion(new Question(
    "Какое произведение написал Лев Толстой?",
    answers7,
    0, // правильный ответ: Война и мир (индекс 0)
    15
));

List<String> answers8 = new ArrayList<>();
answers8.add("Александр Пушкин");
answers8.add("Михаил Лермонтов");
answers8.add("Николай Гоголь");
answers8.add("Федор Достоевский");
literatureTask.addQuestion(new Question(
    "Кто написал 'Евгения Онегина'?",

```

```

        answers8,
        0, // правильный ответ: Александр Пушкин (индекс 0)
        15
    ));

    taskList.add(literatureTask);

    logger.info("Successfully initialized " + taskList.size() + "
tests");
}

public void addTask(Task task){
    taskList.add(task);
}

public void runTest(){
    System.out.println("Система тестов");
    logger.info("Test system starts");

    if(taskList.isEmpty()){
        System.out.println("Нет подходящих тестов");
        logger.warn("There are no tests");
        return;
    }

    System.out.println("Доступные тесты:");
    for (int i = 0; i < taskList.size(); i++){
        System.out.println((i+1) + ". " + taskList.get(i).getTaskName()
            + " (" + taskList.get(i).getQuestions().size()
            + " вопросов)");
    }

    System.out.println("Выберете номер теста, который вы хотите пройти");
    int choice;
    try{
        choice = Integer.parseInt(scanner.nextLine()) - 1;
        if(choice < 0 || choice >= taskList.size()) {
            System.out.println("Вы ввели некорректное число!");
            return;
        }
    }catch (Exception ex){
        logger.error("The entered character is not a number!");
        System.out.println("Введенный символ не является числом!");
        return;
    }

    Task selectedTask = taskList.get(choice);
    takeTask(selectedTask);
}

private void takeTask(Task task){
    System.out.println("Тест по теме: " + task.getTaskName());
    System.out.println("Всего вопросов: " + task.getQuestions().size());
    System.out.println("Максимальный балл: " + task.getTotalScore());
    logger.info("User started doing test on the topic: " +
task.getTaskName() );

    task.resetUserScore();
    List<Question> questions = task.getQuestions();

    for(int i = 0; i < questions.size(); i++){
        Question q = questions.get(i);

```

```

        System.out.println("Вопрос номер: " + (i+1) + " (баллов: " +
q.getPoints() + ")");
        System.out.println(q.toString());

        System.out.println("Введите ваш вариант ответа: ");
        int choice = Integer.parseInt(scanner.nextLine());

        if(q.checkAnswer(choice-1)){
            logger.info("User choose right answer number: " + choice + ".
User gets " + q.getPoints() + " points");
            System.out.println("Верно " + q.getPoints() + " баллов");
            task.addToUserScore(q.getPoints());
        }else{
            logger.warn("User choose incorrect answer: " + choice + ".
Correct answer: " + (q.getRightAnswer()+1));
            System.out.println("Неверно. Правильный ответ: " +
(q.getRightAnswer() + 1));
        }

    }

    showResults(task);

}

public void showResults(Task task){
    System.out.println("Результаты теста: ");
    System.out.println("Тест: " + task.getTaskName());
    System.out.println("Пользователь набрал: " + task.getUserScore() + "
баллов из " + task.getTotalScore());
    logger.info("User get " + task.getUserScore() + " from " +
task.getTotalScore());

    double result = (double) task.getUserScore()/task.getTotalScore() *
100;
    System.out.println("Процент правильных ответов составляет: " + result
+ "%");

    if(result >= 85){
        System.out.println("Оценка отлично!");
        logger.info("User get excellent mark");
    }else if (result >= 70){
        System.out.println("Оценка хорошо!");
        logger.info("User get good mark");
    }else if (result >= 50){
        System.out.println("Оценка удовлетворительно!");
        logger.info("User get normal mark");
    }else {
        System.out.println("Оценка плохо!");
        logger.info("User get bad mark");
    }
}

public void showAllTasks(){
    System.out.println("Список всех тестов:");

    if(taskList.isEmpty()){
        System.out.println("Нет доступных тестов");
        return;
    }

    for(Task task : taskList){
        System.out.println("\n Тест на тему: " + task.getTaskName());
        System.out.println("Всего вопросов: " +

```

```

task.getQuestions().size());
        System.out.println("Максимальное количество баллов за тест: " +
task.getTotalScore());

        for(int i = 0; i < task.getQuestions().size(); i++){
            Question q = task.getQuestions().get(i);

            System.out.println((i+1) + ". " + q.getQuestion() + "
(баллов: " + q.getPoints() + ")");
        }
    }
}

public void close(){
    scanner.close();
}
}

```

Класс `App` содержит точку входа в программу (метод `main`) и отвечает за запуск приложения, инициализацию основных компонентов и организацию главного пользовательского цикла. Он создает логгер `Log4j` для фиксации начала и завершения работы программы, инстанцирует объект `TaskManager`, который управляет всеми тестами, и инициализирует `Scanner` для обработки пользовательского ввода. Основная функция класса — реализация бесконечного цикла с текстовым меню, предлагающим пользователю три варианта действий: пройти тест (вызов `taskManager.runTest()`), просмотреть список всех доступных тестов (`taskManager.showAllTasks()`) или завершить работу. При выборе последнего пункта программа корректно закрывает все используемые ресурсы (`taskManager.close()` и `scanner.close()`) и логирует завершение работы. Класс выступает в роли фасада, обеспечивающего простое и понятное взаимодействие пользователя с системой тестирования, делегируя при этом всю основную логику специализированному классу `TaskManager`.

```

package org.example;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

import java.util.Scanner;

public class App
{
    public static void main( String[] args ) {
        Logger logger = LogManager.getLogger(App.class);
        logger.info("Program starts");
        TaskManager taskManager = new TaskManager();
        Scanner scanner = new Scanner(System.in);

        while(true){
            System.out.println("1. Пройти тест");
            System.out.println("2. Показать все тесты");
            System.out.println("3. Выйти");
            System.out.print("Выберите действие: ");

```

```

        String choice = scanner.nextLine();

        switch (choice){
            case "1" :
                taskManager.runTest();
                break;
            case "2" :
                taskManager.showAllTasks();
                break;
            case "3":
                System.out.println("Конец работы");
                taskManager.close();
                scanner.close();
                logger.info("Program ended");
                return;
            default:
                System.out.println("Неверный выбор! Попробуйте снова.");
        }
    }
}
}
}

```

На следующем рисунке представлен результат работы консольного приложения.

```

D:\TOOLS\Java\bin\java.exe "-javaagent:D:\TOOLS\Idea\IdeaUltimate\IntelliJ IDEA
22:26:41.400 INFO  org.example.App - Program starts
22:26:41.426 INFO  org.example.TaskManager - Initializing test questions
22:26:41.448 INFO  org.example.TaskManager - Successfully initialized 3 tests
1. Пройти тест
2. Показать все тесты
3. Выйти
Выберите действие:

```

Рисунок 1 – результат работы консольного приложения

Вывод: в ходе выполнения лабораторной работы выполнена разработка консольного приложения на языке Java, которое проводит тестирование с подсчетом баллов, а также: ввод ответов, сравнение с правильными, результат прохождения теста(в процентах и статус). В процессе разработки были освоены ключевые принципы создания структурированных консольных приложений.

В процессе работы закреплены следующие знания:

1. Освоена структура консольного приложения Java с разделением кода на классы согласно их функциональному назначению.
2. Реализован ввод и вывод данных через консоль.
3. Применена обработка исключений с помощью конструкции try-catch-finally, что позволило скорректировать реакцию на различные ошибки.
4. Подключена и настроена библиотека Log4j через Maven сборку, а также конфигурация записи данных в консоль и файл.

5. Отточена работа с массивами данных.

Приложение полностью соответствует требованиям лабораторной работы, обладает устойчивостью к некорректному вводу данных, обеспечивает информативное логирование и имеет чёткую структуру кода.